



Emit Telemetry Data to an OpenTelemetry-Compatible Monitoring Tool

Version 2026.1
2026-04-20

Emit Telemetry Data to an OpenTelemetry-Compatible Monitoring Tool

PDF generated on 2026-04-20

InterSystems IRIS Version 2026.1

Copyright © 2026 InterSystems Corporation

All rights reserved.

InterSystems®, HealthShare Care Community®, HealthShare Unified Care Record®, InterSystems IRIS® IntegratedML®, InterSystems Caché®, InterSystems Ensemble®, InterSystems HealthShare®, InterSystems IRIS®, InterSystems IRIS® for Health, and TrakCare are registered trademarks of InterSystems Corporation. HealthShare® Health Connect Cloud™, InterSystems® Data Fabric Studio™, and InterSystems Supply Chain Orchestrator™ are trademarks of InterSystems Corporation. TrakCare is a registered trademark in Australia and the European Union.

All other brand or product names used herein are trademarks or registered trademarks of their respective companies or organizations.

This document contains trade secret and confidential information which is the property of InterSystems Corporation, One Congress Street, Boston, MA 02114, or its affiliates, and is furnished for the sole purpose of the operation and maintenance of the products of InterSystems Corporation. No part of this publication is to be used for any other purpose, and this publication is not to be reproduced, copied, disclosed, transmitted, stored in a retrieval system or translated into any human or computer language, in any form, by any means, in whole or in part, without the express prior written consent of InterSystems Corporation.

The copying, use and disposition of this document and the software programs described herein is prohibited except to the limited extent set forth in the standard software license agreement(s) of InterSystems Corporation covering such programs and related documentation. InterSystems Corporation makes no representations and warranties concerning such software programs other than those set forth in such standard software license agreement(s). In addition, the liability of InterSystems Corporation for any losses or damages relating to or arising out of the use of such software programs is limited in the manner set forth in such standard software license agreement(s).

THE FOREGOING IS A GENERAL SUMMARY OF THE RESTRICTIONS AND LIMITATIONS IMPOSED BY INTERSYSTEMS CORPORATION ON THE USE OF, AND LIABILITY ARISING FROM, ITS COMPUTER SOFTWARE. FOR COMPLETE INFORMATION REFERENCE SHOULD BE MADE TO THE STANDARD SOFTWARE LICENSE AGREEMENT(S) OF INTERSYSTEMS CORPORATION, COPIES OF WHICH WILL BE MADE AVAILABLE UPON REQUEST.

InterSystems Corporation disclaims responsibility for errors which may appear in this document, and it reserves the right, in its sole discretion and without notice, to make substitutions and modifications in the products and practices described in this document.

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)

Tel: +1-617-621-0700

Tel: +44 (0) 844 854 2917

Email: support@InterSystems.com

Table of Contents

Emit Telemetry Data to an OpenTelemetry-Compatible Monitoring Tool.....	1
1 Configure the Target Endpoint	1
2 Emit Metrics	2
3 Emit Logs	2
4 Emit Traces	3
4.1 Optional: Enable Automatic Emission of Traces for Web Application Requests	3
4.2 Instrument Your Application Code	4
4.3 Implement a Custom Sampling Algorithm	9
4.4 Deactivate Tracing	10
5 Error Handling and Recovery	10

Emit Telemetry Data to an OpenTelemetry-Compatible Monitoring Tool

[OpenTelemetry](#) (OTel) is an open source framework and toolkit for generating, exporting, and collecting telemetry data.

On supported systems, this version of InterSystems IRIS leverages the OpenTelemetry C++ SDK to provide support for exporting and emitting telemetry data as [OpenTelemetry Protocol](#) signals over HTTP (OTLP/HTTP) to the [OpenTelemetry Collector](#) or any other compatible monitoring tool.

Note: This feature is *not* available for macOS, Windows, and AIX systems in this version of InterSystems IRIS.

You can configure InterSystems IRIS to emit the following types of signals:

- [Metrics](#) — the measurements which you have configured the InterSystems IRIS [/api/monitor API](#) to collect.
- [Logs](#) — events which InterSystems IRIS records to either the [system messages log](#) or the [audit database](#).
- [Traces](#) — information about how a request moves through your application.

To learn more about how these different types of signals work within OTel, refer to the OTel documentation's pages for [metrics](#), [logs](#), and [traces](#).

InterSystems IRIS pushes these signals to the endpoint that you specify, as described in [Configure the Target Endpoint](#). InterSystems IRIS emits metrics and logs at a regular interval, based on a common configuration parameter; otherwise, you can enable and configure the emission of each type of signal independently, as described in the corresponding sections which follow.

Tip: The OpenTelemetry Collector provides [various debugging tools](#) that can help you as you configure your InterSystems IRIS instance to emit signals, regardless of the monitoring tool which you ultimately choose to deploy within your production system.

1 Configure the Target Endpoint

To specify the endpoint to which an InterSystems IRIS instance sends OTLP/HTTP signals, set the environment variable `OTEL_EXPORTER_OTLP_ENDPOINT` on the instance's host system to the desired address. For instructions on setting environment variables, refer to your operating system's documentation. You can confirm that InterSystems IRIS can access the environment variable by invoking the `GetEnviron()` utility method from a Terminal session.

If you enable OTLP/HTTP emission and an `OTEL_EXPORTER_OTLP_ENDPOINT` environment variable is not set, the instance emits signals to the default endpoint for the [OpenTelemetry Collector](#)'s OTLP/HTTP receiver: `http://localhost:4318`.

2 Emit Metrics

InterSystems IRIS can emit all of the metric events that the [/api/monitor API](#) collects (including your [custom application metrics](#)) to the OTLP/HTTP endpoint that you designate, at regular intervals.

To configure your instance to emit metric events:

1. Configure the instance to collect all of the metrics that you want to collect. If you want to collect interoperability production metrics, you must [manually enable them](#). You can also configure the instance to collect [custom application metrics](#).
2. Configure your instance's OpenTelemetry exporter to start emitting metrics when the instance starts. You can do this in any of the following ways:
 - From the [Management Portal](#): navigate to **System Administration > Configuration > Additional Settings > Monitor**, and select **Enable OTel Metrics**. Then, select **Save**.
 - Change the [OTELMetrics](#) parameter by modifying the Config.Monitor class (as described in the class reference) or by [editing the CPF file](#) directly.
3. If needed, modify the frequency at which the exporter will emit metrics. By default, the exporter emits signals every 10 seconds. You can change this interval in any of the following ways:
 - From the Management Portal: navigate to **System Administration > Configuration > Additional Settings > Monitor**. Update the **OTel Exporter Interval** field with the length of the desired interval, in seconds. Then, select **Save**.
 - Change the [OTELInterval](#) parameter by modifying the Config.Monitor class (as described in the class reference) or by [editing the CPF file](#) directly.
4. If you configured your instance in the preceding steps by modifying CPF parameters, restart the instance to allow your changes to take effect.

3 Emit Logs

InterSystems IRIS can emit OTLP/HTTP signals for the same categories of log events which would be part of a [structured log](#) file—namely, events which are recorded to the [system messages log](#) (messages.log) or to the [audit database](#). InterSystems IRIS emits structured log events to the OTLP/HTTP endpoint that you designate, at regular intervals.

To configure your instance to emit log events:

1. Configure your instance's OpenTelemetry exporter to start emitting log events when the instance starts. You can do this in any of the following ways:
 - From the [Management Portal](#): navigate to **System Administration > Configuration > Additional Settings > Monitor**, and select **Enable OTel Logs**. Then, select **Save**.
 - Change the [OTELLogs](#) parameter by modifying the Config.Monitor class (as described in the class reference) or by [editing the CPF file](#) directly.
2. As needed, configure the minimum severity level that a log event must meet or exceed in order to be emitted by the instance's OpenTelemetry exporter. Severity levels are the same as those [used in the structured log](#). The default severity level threshold is WARN. At this level, the exporter emits log events from the WARN, SEVERE, and FATAL levels; it does not emit log events from the DEBUG2, DEBUG, and INFO levels.

You can change the minimum severity level in any of the following ways:

- From the Management Portal: navigate to **System Administration > Configuration > Additional Settings > Monitor**. Select the desired threshold severity level from the **OTel Log Level** drop-down menu. Then, select **Save**.
 - Change the [OTELLogLevel](#) parameter by modifying the Config.Monitor class (as described in the class reference) or by [editing the CPF file](#) directly.
3. If needed, modify the frequency at which the exporter will emit log events. By default, the exporter emits signals every 10 seconds.

You can change this interval in any of the following ways:

- From the Management Portal: navigate to **System Administration > Configuration > Additional Settings > Monitor**. Update the **OTel Exporter Interval** field with the length of the desired interval, in seconds. Then, select **Save**.
 - Change the [OTELInterval](#) parameter by modifying the Config.Monitor class (as described in the class reference) or by [editing the CPF file](#) directly.
4. If you configured your instance in the preceding steps by modifying CPF parameters, restart the instance to allow your changes to take effect.

4 Emit Traces

A trace records how a request moves through an application. It consists of one or more nestable spans, which represent the constituent units of work that the application performs as part of responding to the request. To record a trace, the application's code must include instruments which generate spans, populate them with information, and contextualize them as part of a continuous trace. For more information about the OpenTelemetry specification for traces, refer to the [OpenTelemetry documentation](#).

4.1 Optional: Enable Automatic Emission of Traces for Web Application Requests

InterSystems IRIS can automatically generate and emit traces for each request that a CSP, SOAP, or REST web application sends. To enable automatic trace emission for a web application, perform the following steps:

1. Access the Management Portal for the InterSystems IRIS instance which hosts your web application.
2. Navigate to **System Administration > Security > Applications > Web Applications**. If necessary, select **Go** to access the **Web Applications** page.
3. From the **Web Application** table, select the **Name** of the web application for which you wish to enable tracing.
4. The **Enable** section of the **Edit Web Application** page provides an **Enable Traces** checkbox for both **REST** and **CSP/ZEN** applications. Select the **Enable Traces** checkbox which is appropriate to your web application.
5. Select **Save**.

Note: The **Enable Traces** checkbox corresponds to the TraceEnabled property of the Security.Applications class. When necessary, you can enable or disable automatic tracing for a web application programmatically by modifying this property.

If you have created a `TracerProvider` object for the web application's namespace at startup using `SetTracerProvider()` as described in the [instrumentation instructions](#), InterSystems IRIS uses that `TracerProvider` to generate automatic web application traces. Otherwise, InterSystems IRIS uses a `TracerProvider` with default attributes.

For each request, the `TracerProvider` instantiates a `Tracer` object which is named after the web application. This `Tracer` emits a span which is named after the class that is dispatching the request (for CSP and SOAP requests) or the endpoint URL (for REST requests).

To populate an outbound request's `traceparent` and `tracestate` headers with information about your application's current active span, employ one of the injection methods that the `%Trace.Propagation` class provides, as described in the instrumentation instructions.

4.2 Instrument Your Application Code

Within InterSystems IRIS, the `%Trace` package provides a straightforward API for instrumenting your application code to produce traces in a way that conforms to the OTel specification. Once instruments within your application are producing traces, InterSystems IRIS emits them to the OTLP/HTTP endpoint that you designate.

In a [supported IDE](#), edit your application code to produce traces using the `%Trace` API as follows:

1. Optional: if you want to [sample](#) a subset of your application's traces, create an instance of a sampler class that extends `%Trace.AbstractSampler`. You can implement your own sampling algorithm for your application by defining a custom `ShouldSample()` method. Refer to [Implement a Custom Sampling Algorithm](#) for further details.

To help you get started, the `%Trace` package provides the following sampler classes, which have the following effects:

- `%Trace.AlwaysOffSampler` — the application does not record or emit any spans.
- `%Trace.AlwaysOnSampler` — the application records and emits every span.
- `%Trace.TraceIdRatioBasedSampler` — the application records and emits a fixed percentage of traces as specified by the sampler class's `Ratio` property, which can assigned a decimal value between 0 and 1. To decide whether to sample a span, the sampler converts a hexadecimal interpretation of the rightmost characters of the span's trace ID to a decimal number value and then compares that value against a threshold derived from the specified `Ratio`.
- `%Trace.ParentBasedSampler` — the application conditionally applies logic from different sampler classes to determine whether a span should be recorded and emitted, depending upon the following conditions:
 - whether the span is the root span of a trace or a child span.
 - whether the span's parent was initialized locally or remotely.
 - whether the span's parent was sampled or not.

The following example creates an instance of the `%Trace.TraceIdRatioBasedSampler` which configures the application to record and emit 10% of the traces that it creates.

ObjectScript

```
set sampleratio = 0.1
set sampler = ##class(%Trace.TraceIdRatioBasedSampler) .%New(sampleratio)
```

Python

```
sampleratio = 0.1
sampler = iris.cls('%Trace.TraceIdRatioBasedSampler')._New(sampleratio)
```

2. Create an instance of the `%Trace.TracerProvider` class to serve as your application's [Tracer Provider](#).

The constructor for this class accepts two optional arguments. The first argument—passed by reference—is an array of key-value pairs which is used to set the `TracerProvider` object's `ResourceAttributes` property. You can use this array

to specify global attributes about your application. If you have defined a sampler object for your application, provide it to the constructor as its second argument. Any traces that the application generates using this `TracerProvider` will be sampled according to the algorithm described by the specified sampler object.

The following example instantiates a `TracerProvider` (`tracerProv`) with a resource array identifying the name and version number of the service that it is tracing. Any traces which `tracerProv` creates will be sampled according to the logic described by `sampler`.

ObjectScript

```
set attributes("service.name") = "test_service"
set attributes("service.version") = "2.0"
set tracerProv = ##class(%Trace.TracerProvider).%New(.attributes, sampler)
```

Python

```
attributes = {}
attributes["service.name"] = "test_service"
attributes["service.version"] = "2.0"
attrArray = iris.arrayref(attributes)
tracerProv = iris.cls('%Trace.TracerProvider')._New(.attrArray, sampler)
```

Note: If you have not specified a sampler object when you create a `TracerProvider`, InterSystems IRIS records and emits every span that was created by Tracer objects which that `TracerProvider` generated. However, it does not set the `TraceFlags` or `TraceState` properties for a span's context.

3. In most situations, it is preferable to instantiate a single `TracerProvider` object at startup, for shared use by instrumentation code across the namespace throughout the entire life cycle of the application. To designate a `TracerProvider` object for use across a namespace, provide it to the **`SetTracerProvider()`** method of the `%Trace.Provider` class, as follows:

ObjectScript

```
do ##class(%Trace.Provider).SetTracerProvider(tracerProv)
```

Python

```
iris.cls('%Trace.Provider').SetTracerProvider(tracerProv)
```

When you need to access the `TracerProvider` object within your instrumentation code (as described in later steps), use the complementary **`GetTracerProvider()`** method to recall it:

ObjectScript

```
set tracerProv = ##class(%Trace.Provider).GetTracerProvider()
```

Python

```
tracerProv = iris.cls('%Trace.Provider').GetTracerProvider()
```

4. To instrument your application code, first use the `TracerProvider` object's `GetTracer()` method to instantiate a `Tracer`. (The `Tracer` object generates the actual spans of the trace.)

`GetTracer()` accepts two arguments, *Name* and *Version*. Use these arguments to uniquely identify the application or application component that you are tracing and specify its version number. For example:

ObjectScript

```
set tracer = tracerProv.GetTracer("service.orderprocessor", "2.0.2")
```

Python

```
tracer = tracerProv.GetTracer("service.orderprocessor", "2.0.2")
```

5. Start a root span using the Tracer object's `StartSpan()` method. In order, **StartSpan()** accepts the following arguments, which are used to define the properties of the span:
 - a. *Name* — A string, used to set the span's name field
 - b. *Parent* — (Optional.) A `%Trace.Context` object that identifies the span which you wish to specify as the parent span for the new span. If you do not provide a *Parent* and you have not previously specified an active span (as described in a later step), then the method initializes the span as a root span, with a unique Trace ID.
 - c. *Spankind* — (Optional.) A string, identifying the span as belonging to one of the OpenTelemetry specification's recognized [span kinds](#). If you do not provide a *Spankind*, the span is classified as `Internal` by default.
 - d. *Attributes* — (Optional.) An array of key-value pairs, passed by reference.
 - e. *StartTime* — (Optional.) A timestamp recording the span's start time, in `$ZTIMESTAMP` format. If not provided, *StartTime* is set to the current time.

For example, code which initializes a root span for processing a retail transaction may resemble the following:

ObjectScript

```
set rootAttr("customer.id") = customer.ID
set rootAttr("product.id") = product.ID
set rootSpan = tracer.StartSpan("order", , "Server", .rootAttr)
```

Python

```
rootspan = {}
rootAttr("customer.id") = customer.ID
rootAttr("product.id") = product.ID
rootAttrArray = iris.arrayref(rootAttr)
set rootSpan = tracer.StartSpan("order", , "Server", .rootAttrArray)
```

Note: If you want your application to propagate trace context from a web request that it has received into a new span, use the `HttpExtract()` or `CSPRequestExtract()` method of the `%Trace.Propagation` class to enhance the context of the current active span with the context provided by an HTTP header string or the headers of a `%CSP.Request` object (respectively). You can then specify this context object as the parent of the new span when you invoke **StartSpan()**. The following example demonstrates this procedure using a hypothetical CSP request, *cspRequest*:

ObjectScript

```
set currentContext = ##class(%Trace.Context).GetCurrentContext()
set requestParentContext = ##class(%Trace.Propagation).CSPRequestExtract(cspRequest,
currentContext)
set restApiSpan = tracer.StartSpan("rest_function", requestParentContext, "Server")
```

Python

```
currentContext = iris.cls('%Trace.Context').GetCurrentContext()
requestParentContext = iris.cls('%Trace.Propagation').CSPRequestExtract(cspRequest,
currentContext)
restApiSpan = tracer.StartSpan("rest_function", requestParentContext, "Server")
```

6. As needed, nest child spans hierarchically within this root span. For simple implementations, you can manually specify the parent span for a new span by creating a `%Trace.Context` object which identifies the desired parent as the *ActiveSpan*, and then providing that Context object as the *Parent* argument of **StartSpan()**.

However, attempting to manage context across lexical scopes using this manual approach would be impractical. For this reason, the `%Trace` API provides a dynamic scoping mechanism for managing context.

To manage the distributed context of your trace dynamically, perform the following steps:

- a. Designate the parent span as the "active" span using the Tracer object's `SetActiveSpan()` method, as follows:

ObjectScript

```
set rootScope = tracer.SetActiveSpan(rootSpan)
```

Python

```
rootScope = tracer.SetActiveSpan(rootSpan)
```

SetActiveSpan() returns an instance of the `%Trace.Scope` class. This Scope object acts as a record that the corresponding span is active.

- b. Start a new span by invoking **StartSpan()** *without* specifying a *Parent*, as follows:

ObjectScript

```
set childSpan1 = tracer.StartSpan("order_payproc")
```

Python

```
childSpan1 = tracer.StartSpan("order_payproc")
```

As long as the active span's Scope object remains in memory, **StartSpan()** initializes new spans as children of the active span by default when no other *Parent* is specified.

- c. To nest spans further, invoke **SetActiveSpan()** on a child span to designate it as the new active span and generate a new Scope object. The active span is identified by the newest Scope object which exists in memory at a given time. Therefore, once you have generated a Scope object for a new active span, **StartSpan()** will initialize new spans as its children by default.

Continuing the previous example, the following code starts a new span *childSpan2* as a child of *childSpan1* (which is itself a child of *rootScope*):

ObjectScript

```
set child1Scope = tracer.SetActiveSpan(childSpan1)
set childSpan2 = tracer.StartSpan("order_payproc_addnewcard")
```

Python

```
child1Scope = tracer.SetActiveSpan(childSpan1)
childSpan2 = tracer.StartSpan("order_payproc_addnewcard")
```

- d. When you destroy the Scope object for the current active span, the span which was previously active becomes the default parent for **StartSpan()** once again (assuming you have not destroyed its Scope object as well). Continuing the previous examples, the following code starts a new span *childSpan3* as a child of *rootScope* and a sibling of *childSpan1*:

ObjectScript

```
kill child1Scope
set childSpan3 = tracer.StartSpan("order_sendconfirm", , "Server")
```

Python

```
iris.execute('kill child1Scope')
childSpan3 = tracer.StartSpan("order_sendconfirm", , "Server")
```

7. As needed, define information about your spans. Available methods for this purpose include the following:

- Use the Span object's `AddEvent()` method to add a [span event](#).
- Use the Span object's `AddLink()` method to add a [span link](#).
- Modify the `TraceFlags` and `TraceState` properties of the Span object's `Context` property (an instance of `%Trace.SpanContext`) to modify a span's trace flags and trace state, respectively.

Continuing the previous examples, the following code enhances the "order_payproc" span (*childSpan1*) with a "paymentdeclined" event and a link to a hypothetical span named *paymentProcLivenessSpan*:

ObjectScript

```
set eventAttr("declined.reason")="Unknown error occurred."
do childSpan1.AddEvent("paymentdeclined", .eventAttr)
do childSpan1.AddLink(paymentProcLivenessSpan.Context)
```

Python

```
eventAttr = {}
eventAttr("declined.reason") = "Unknown error occurred."
eventAttrArray = iris.arrayref(eventAttr)
childSpan1.AddEvent("paymentdeclined", .eventAttrArray)
childSpan1.AddLink(paymentProcLivenessSpan.Context)
```

8. If you are sending an HTTP request from your application, invoke an injection method from the `%Trace.Propagation` class to propagate context into the request's headers. If you are constructing the HTTP request as a `%Net.HttpRequest` object, use the `HttpRequestInject()` method to inject `traceparent` and `tracestate` headers into the request object automatically. For other use cases, the `HttpInject()` method accepts a string containing the headers for an HTTP request and appends the aforementioned trace context headers to it.

Assuming a hypothetical `%Net.HttpRequest` object (*request*), the following code would retrieve the `%Trace.Context` object for the current active span and inject `traceparent` and `tracestate` headers into *request* based on that context object. It would then dispatch the enhanced request object as an HTTP GET request.

ObjectScript

```
set currentcontext = ##class(%Trace.Context).GetCurrentContext()
do ##class(%Trace.Propagation).HttpRequestInject(request, currentcontext)
do request.Get("/payment/auth")
```

Python

```
currentcontext = iris.cls('%Trace.Context').GetCurrentContext()
iris.cls('%Trace.Propagation').HttpRequestInject(request, currentcontext)
request.Get("/payment/auth")
```

9. Before you end a span, update the span's status using the Span object's `SetStatus()` method, as in the following example:

ObjectScript

```
do childSpan1.SetStatus("Ok")
```

Python

```
childSpan1.SetStatus("Ok")
```

SetStatus() accepts one argument (a string) which can have three possible values corresponding to the three [span statuses](#) recognized by the OpenTelemetry specification.

10. End each span using the Span object's `End()` method. **End()** accepts an optional argument: a timestamp, in **\$TIMESTAMP** format. If a timestamp is provided, **End()** records that time as the end time for the span. Otherwise, **End()** sets the span's end time to the current time, as in the following example:

ObjectScript

```
do childSpan1.End()
```

Python

```
childSpan1.End()
```

InterSystems IRIS invokes the OpenTelemetry SDK to export the span when you end it, assuming that the ‘sampled’ bit in the span Context property’s TraceFlags has been set appropriately.

11. If you are using **SetActiveSpan()** to manage nested spans across lexical scopes (as suggested in a preceding step), destroy the Scope object for each active span after you end the span. Continuing the previous examples, the following code would conclude the trace encompassed by *rootSpan* and prepare the application to record a new trace:

ObjectScript

```
do rootSpan.SetStatus("Ok")
do rootSpan.End()
kill rootScope
```

Python

```
rootSpan.SetStatus("Ok")
rootSpan.End()
iris.execute('kill rootScope')
```

12. As needed, import and recompile the code you have edited to enable tracing for your application.

4.3 Implement a Custom Sampling Algorithm

As mentioned by the [instrumentation instructions](#), you can sample a subset of your application’s spans according to a custom sampling algorithm by defining a custom sampler class.

In a [supported IDE](#), create a new class within the namespace for your application. This class should extend `%Trace.AbstractSampler`, and it must override the superclass’s **ShouldSample()** method with code that evaluates a span’s context according to your desired logic. When you invoke the TracerProvider for your application with a sampler object specified, the **StartSpan()** method of any Tracer object which the TracerProvider generates will invoke the sampler’s **ShouldSample()** method to determine whether to sample the new span.

The **ShouldSample()** method should accept the following arguments:

- *Context* — a `%Trace.SpanContext` object which contains information about the new span’s parent.
- *TraceId* — the trace ID associated with the new span’s parent span.
- *Name* — the name you have specified for the new span.
- *SpanKind* — the new span’s span kind.
- *Attributes* — the optional array of key-value pairs which the **StartSpan()** method received. You must pass this argument to the method by reference.

Given this information, the **ShouldSample()** method must return an instance of `%Trace.SamplingResult`. The application samples the new span based on the Decision property, which can have one of three values:

- "DROP" instructs the application to discard the span.
- "RECORD_ONLY" instructs the application to record the span, so that your application can still access the span object and its contents. However, InterSystems IRIS will not emit the span to the OTLP/HTTP endpoint when it ends.

- "RECORD_AND_SAMPLE" instructs the application to record the span and emit it to the OTLP/HTTP endpoint when it ends.

The `%Trace.SamplingResult` that your **ShouldSpan()** method returns should maintain or update the trace state for the span using the `TraceState` property. You can also use its `Attributes` property to add attributes to the `Attributes` array for the new span.

The `%Trace` package provides several sampler classes to assist you as you implement your desired sampling algorithm. The `%Trace.ParentBasedSampler` class is especially noteworthy: it allows you to define multiple custom sampler classes and apply them conditionally to evaluate spans depending upon whether they are root spans or child spans, where their parent spans were created, and whether their parent spans were sampled. The reference for the class describes the conditions which are available (these are mapped as properties within the class). The reference for the `%OnNew()` method describes the order in which you should supply these instances to the constructor when you create an instance of the `%Trace.ParentBasedSampler` within your application code. You would then specify this instance of the `%Trace.ParentBasedSampler` class as the sampler for your application's `TracerProvider`.

4.4 Deactivate Tracing

To deactivate tracing for your application after you have instrumented it, perform the following steps:

1. Edit your code so that it initializes any Tracer Provider that your application uses as an instance of `%Trace.NoopTracerProvider` (*instead of* `%Trace.TracerProvider`). Revisiting the example provided in the preceding [instrumentation instructions](#), deactivation would require you to change the line of code which sets the `tracerProv` object so that it reads as follows:

ObjectScript

```
set tracerProv = ##class(%Trace.NoopTracerProvider).%New(.attributes)
```

Python

```
tracerProv = iris.cls('%Trace.NoopTracerProvider')._New(.attrArray)
```

Assuming that `tracerProv` has been set as the Tracer Provider which serves the entire namespace, no further edits would be necessary.

2. As needed, import and recompile the code you have edited to deactivate tracing for your application.

5 Error Handling and Recovery

An InterSystems IRIS instance's OpenTelemetry exporter maintains a log file which you can use to troubleshoot any errors that you may encounter while you are configuring your instance to emit signals. The path to the log file within the instance's installation directory (`<installDir>`) is `<installDir>/mgr/irisotel.log`.

Additionally, if the OpenTelemetry-compatible tool which was receiving signals at the OTLP/HTTP endpoint becomes unavailable due to an unexpected system error, the instance's OpenTelemetry exporter logs an error to the system messages log (`messages.log`). It then stops emitting signals from the instance.

The `SYS.Monitor.OTel` class provides methods to help you test whether communication at the OTLP/HTTP endpoint has been restored: `TestLogs()`, `TestMetrics()`, and `TestTraces()`.

Once you have resolved the cause of the error and restored the OTLP/HTTP connection, you can resume the emission of signals as follows:

1. Open a Terminal session on the instance and navigate to the %SYS namespace.
2. To resume the emission of metrics and logs, execute the following command:

Terminal

```
do ##class(SYS.Monitor.OTel).Start()
```

3. To resume the emission of traces, execute the following command:

```
do ##class(SYS.Monitor.OTel).EnableTraces()
```

